ALGORITHMS LABORATORY

Theory and Exercises explained step by step

Author: SamuMGH04

WELCOME

Thanks for choosing these notes. This project is created with the aim to go more deep in C++ programming and explore advanced concepts.

LICENSE

These works are protected and distributed under the license
Creative Commons Attribution-NonCommercial-NoDerivatives 4.0 International

- **Attribution:** You must mention the original creator
- **Non Commercial:** You can't sell this material
- **No Derivatives:** You can't modify or change the text.

SOVRACCARICAMENTO (OVERLOADING) DEGLI OPERATORI

Indice

1 Introduzione all'Overloading degli Operatori **3**

1.1 Finalità e Vantaggi dell'Overloading 3

1.2 Analisi Comparativa: Tipi Built-in vs Tipi Custom 3

1.3 Implementazione Pratica: Un Esempio Strutturato 4

1.3.1 Spiegazione Passo-Passo del Codice 5

2 Classificazione degli Operatori Sovraccaricabili **6**

2.1 Operatori Aritmetici 6

2.2 Operatori di Confronto (Relazionali) 6

2.3 Operatore di Stream (I/O) 7

2.4 Implementazione Didattica: La Classe `rational` 7

2.4.1 Spiegazione Passo-Passo delle Operazioni 8

3 La Natura Funzionale degli Operatori **9**

3.1 Sintassi e Dichiarazione 9

3.2 Esempio: Implementazione dell'Addizione tra Razionali 9

3.2.1 Spiegazione Passo-Passo del Codice 10

3.3 Osservazione sulla Risoluzione delle Chiamate 11

4 Perché Sovraccaricare gli Operatori? **12**

4.1 Confronto: Funzioni vs Operatori 12

4.1.1 Approccio Standard (Senza Overloading) 12

4.1.2 Approccio con Overloading 12

4.2 Obiettivi Didattici e Professionali 13

4.3 Esempio di Codice Comparativo 13

5 Operatori Membri vs Operatori Non-Membri **15**
5.1 Operatori come Funzioni Membro 15
5.2 Operatori come Funzioni Non-Membri (Libere) 15
5.3 Operatori di Confronto 15
5.4 Implementazione Tecnica e Confronto 16
5.4.1 Analisi Passo-Passo 17

6 Buone Pratiche nell'Overloading degli Operatori **18**
6.1 Principi Guida per una Progettazione Corretta 18
6.2 L'Esempio del Tipo Razionale 19
6.3 Esempio di Codice: Implementazione "Safe" e Coerente 19

7 Overloading dell'Operatore di Output << **22**
7.1 Perché l'Operatore di Output deve essere una Funzione Libera? 22
7.2 Analisi della Firma della Funzione 22
7.3 Implementazione Dettagliata 22
7.3.1 Spiegazione Passo-Passo dell'Esecuzione 24

8 Anatomia della Firma dell'Operatore di Output **25**
8.1 Analisi dei Componenti della Firma 25
8.2 Implementazione e Flusso Logico 26
8.2.1 Osservazione sui Prerequisiti 27

9 Perché restituire `std::ostream&`? **28**
9.1 Il Meccanismo del Concatenamento 28
9.2 Conseguenze della Mancata Restituzione 28
9.3 Analisi del Codice e Flusso di Ritorno 29
9.3.1 Spiegazione Passo-Passo dell'Infrastruttura di Ritorno 30

1 Introduzione all'Overloading degli Operatori

Nel linguaggio C++, gli operatori non sono semplici entità sintattiche astratte, ma devono essere interpretati come funzioni speciali. La potenza del C++ risiede nella sua estensibilità: l'utente non è limitato a utilizzare gli operatori predefiniti (come +, -, *, ecc.) solo sui tipi primitivi forniti dal linguaggio (come `int`, `double` o `float`), ma può ridefinirne il comportamento per le proprie classi o strutture.

L'Overloading degli Operatori (sovra-caricamento) è la funzionalità che permette di associare un nuovo significato a un operatore esistente affinché possa operare su tipi definiti dall'utente (*User-Defined Types*).

1.1 Finalità e Vantaggi dell'Overloading

L'adozione dell'overloading degli operatori risponde a tre esigenze fondamentali nella progettazione del software orientato agli oggetti:

- **Definizione del comportamento per tipi personalizzati**: Permette di specificare esattamente cosa accade quando, ad esempio, si sommano due oggetti di una classe complessa (come una matrice o un numero razionale).
- **Uniformità tra tipi**: Consente agli oggetti creati dall'utente di comportarsi con la stessa naturalezza dei tipi *built-in* (predefiniti). Questo riduce il carico cognitivo per il programmatore, che può utilizzare una sintassi familiare.
- **Miglioramento della leggibilità e dell'espressività**: Sostituire chiamate a funzioni prolisse (come `a.somma(b)`) con operatori intuitivi (come `a + b`) rende il codice più pulito, conciso e simile alla notazione matematica standard.

1.2 Analisi Comparativa: Tipi Built-in vs Tipi Custom

Per comprendere l'utilità pratica, confrontiamo il comportamento standard con quello desiderato in un tipo complesso.

- **Tipo Built-in**: L'operazione `3 + 4` è immediatamente compresa dal compilatore, che genera l'istruzione macchina per sommare due interi.
- **Tipo Custom (Esempio Numeri Razionali)**: Consideriamo una classe che rappresenta una frazione, `rational<int>1, 2` (che rappresenta $\frac{2}{1}$ Senza l'overloading, il compilatore non saprebbe come gestire l'espressione:

```
rational<int>1,2 + rational<int>1,3
```

Attraverso l'overloading, possiamo istruire il compilatore affinché esegua il calcolo del minimo comune denominatore e restituisca correttamente la frazione risultante $\frac{6}{5}$

1.3 Implementazione Pratica: Un Esempio Strutturato

Di seguito viene presentato un esempio di codice C++ che illustra come implementare l'overloading dell'operatore di somma per una struttura ipotetica.

Listing 1: Esempio di implementazione dell'Overloading dell'operatore +

```
#include <iostream>

struct Punto {
int x, y; // Coordinate del punto nel piano cartesiano

// Overloading dell'operatore + definito come funzione membro
Punto operator+(const Punto& altro) const {
    // Ritorna un nuovo oggetto Punto sommando le componenti membro a membr
    return Punto{x + altro.x, y + altro.y};
}
};

int main() {
Punto p1{1, 2}; // Inizializzazione del primo oggetto Punto (x=1, y=2)
Punto p2{3, 4}; // Inizializzazione del secondo oggetto Punto (x=3, y=4)

Punto p3 = p1 + p2; // Utilizzo dell'operatore sovraccaricato: p1.operator+

std::cout << "Risultato: " << p3.x << ", " << p3.y << std::endl; // Stampa
return 0; // Fine del programma con successo
}
```

Output Atteso

```
Risultato: 4, 6
```

1.3.1 Spiegazione Passo-Passo del Codice

1. **Definizione della Struttura**: Viene definita la `struct Punto`. In C++, le strutture possono contenere funzioni membro (metodi).

2. **La Firma dell'Operatore**: La riga `Punto operator+(const Punto& altro) const` è il cuore del concetto.

   - `operator+`: È il nome della "funzione speciale". Il prefisso `operator` è obbligatorio.
   - `const Punto& altro`: Passiamo l'operando di destra per riferimento costante per evitare copie inutili di memoria e garantire che l'oggetto originale non venga modificato.
   - `const` (alla fine): Indica che la funzione non modificherà l'oggetto di sinistra (`p1` nell'esempio).

3. **Corpo della Funzione**: Viene creato e restituito "al volo" un nuovo `Punto` dove la coordinata `x` è la somma delle `x`, e lo stesso per `y`.

4. **Utilizzo nel Main**: Quando scriviamo `p1 + p2`, il compilatore traduce internamente questa istruzione in una chiamata di funzione: `p1.operator+(p2)`. Il risultato viene poi assegnato a `p3`.

5. **Output**: La stampa conferma che le componenti sono state sommate correttamente (1+3=4 e 2+4=6).

2 Classificazione degli Operatori Sovraccaricabili

In C++, la flessibilità del linguaggio permette di estendere il comportamento di una vasta gamma di operatori. Questo è particolarmente utile quando si progettano classi che rappresentano entità matematiche o numeriche, dove l'uso di simboli standard rende il codice estremamente più leggibile e vicino al linguaggio naturale del dominio di riferimento.

Prendendo come riferimento una classe numerica per numeri razionali, indicata come `rational<T>`, è possibile individuare tre categorie principali di operatori che solitamente vengono sovraccaricati per garantire che gli oggetti si comportino in modo analogo ai tipi numerici predefiniti (*built-in*).

2.1 Operatori Aritmetici

Gli operatori aritmetici sono fondamentali per definire le operazioni di base tra due istanze di una classe numerica. Per un tipo come `rational<T>`, i simboli da implementare sono:

- `+` (Addizione)
- `-` (Sottrazione)
- `*` (Moltiplicazione)
- `/` (Divisione)

L'overloading di questi operatori consente di scrivere espressioni complesse come `a * b + c / d` invece di annidare chiamate a funzioni come `somma(moltiplica(a, b), dividi(c, d))`.

2.2 Operatori di Confronto (Relazionali)

Affinché una classe sia pienamente funzionale, deve essere possibile confrontare le sue istanze. Questo è essenziale non solo per la logica di business, ma anche per permettere agli oggetti di essere ordinati all'interno dei contenitori della libreria standard (come `std::set` o `std::sort`). Gli operatori includono:

- `==` (Uguaglianza) e `!=` (Disuguaglianza)
- `<` (Minore) e `>` (Maggiore)
- `<=` (Minore o uguale) e `>=` (Maggiore o uguale)

2.3 Operatore di Stream (I/O)

Per facilitare il debug e l'interazione con l'utente, si sovraccarica l'operatore di inserimento nel flusso:

- « (Inserimento in stream di output)

Questo permette di inviare l'oggetto direttamente a `std::cout` (ad esempio, `std::cout « miaFrazione;`) definendo internamente come la frazione debba essere visualizzata (es. "1/2").

2.4 Implementazione Didattica: La Classe `rational`

Vediamo come si traduce quanto esposto in un codice C++ concreto, analizzando la struttura necessaria per supportare queste operazioni.

Listing 2: Esempio di overloading per una classe razionale

```
#include <iostream>

template <typename T>
class rational {
public:
T num, den; // Numeratore e denominatore di tipo generico T

// Costruttore per inizializzare la frazione
rational(T n, T d) : num(n), den(d) {} // Inizializza i membri num e den

// Overloading dell'operatore == (Uguaglianza)
bool operator==(const rational& altro) const {
    // Due frazioni a/b e c/d sono uguali se a*d == c*b
    return (num * altro.den == altro.num * den);
}

// Overloading dell'operatore * (Moltiplicazione)
rational operator*(const rational& altro) const {
    // La moltiplicazione tra frazioni moltiplica numeratori e
    denominatori tra loro
    return rational(num * altro.num, den * altro.den);
}
};
```

```
// Overloading dell'operatore << (Globale per permettere std::cout << obj)
template <typename T>
std::ostream& operator<<(std::ostream& os, const rational<T>& r) {
// Inserisce nel flusso di output la rappresentazione testuale "num/den"
os << r.num << "/" << r.den;
return os; // Ritorna lo stream per permettere il concatenamento (chaining)
}

int main() {
rational<int> r1{1, 2}; // Crea la frazione 1/2
rational<int> r2{2, 4}; // Crea la frazione 2/4 (equivalente a 1/2)

if (r1 == r2) { // Utilizza l'operatore == sovraccaricato
    std::cout << r1 << " e " << r2 << " sono equivalenti." << std::endl;
    // Utilizza << sovraccaricato
}

return 0; // Termina l'esecuzione
}
```

Output Atteso

```
1/2 e 2/4 sono equivalenti.
```

2.4.1 Spiegazione Passo-Passo delle Operazioni

1. **Uguaglianza (`operator==`)**: All'interno della funzione, implementiamo la proprietà matematica dell'uguaglianza tra frazioni tramite il prodotto incrociato. Questo evita problemi di precisione che potrebbero derivare dalla conversione in virgola mobile (`double`).

2. **Moltiplicazione (`operator*`)**: Viene creato un nuovo oggetto `rational` temporaneo. Notare che il tipo di ritorno è `rational` (per valore) e non un riferimento, poiché stiamo creando un nuovo valore numerico che è il risultato dell'operazione.

3. **Operatore di Stream (`operator«`)**: Questo operatore deve essere definito **fuori** dalla classe come funzione libera. Questo perché il primo operando (quello a sinistra del simbolo «) è un oggetto di tipo `std::ostream` e non `rational`. Il ritorno di `std::ostream&` è fondamentale per consentire il "chaining", ovvero l'istruzione `cout « a « b;`.

4. **Esecuzione nel `main`**: Il compilatore risolve `r1 == r2` chiamando il metodo membro di `r1`, mentre risolve `std::cout « r1` cercando la funzione globale definita subito sopra.

Nota didattica: Sebbene quasi tutti gli operatori siano sovraccaricabili, alcuni sono esclusi dal linguaggio per evitare ambiguità sintattiche insormontabili, come l'operatore di risoluzione dell'ambito (`::`), l'operatore punto (`.`), l'operatore ternario (`?:`) e l'operatore `sizeof`.

3 La Natura Funzionale degli Operatori

In C++, un operatore non deve essere considerato come un'entità sintattica isolata, ma come una funzione con un nome speciale. La sintassi che utilizziamo comunemente (come a+b) è in realtà una "scorciatoia" (zucchero sintattico) per una chiamata a funzione.

3.1 Sintassi e Dichiarazione

Per definire un operatore, si utilizza la parola chiave `operator` seguita dal simbolo dell'operatore che si desidera sovraccaricare. La firma di tale funzione segue le regole standard delle funzioni C++, specificando il tipo di ritorno e i parametri in ingresso.

3.2 Esempio: Implementazione dell'Addizione tra Razionali

Consideriamo l'implementazione dell'operatore di somma per la classe `rational<T>`. L'obiettivo è tradurre la regola matematica della somma tra frazioni:

$\frac{a}{b} + \frac{c}{d} = \frac{ad+bc}{bd}$

Listing 3: Implementazione dell'operatore di somma come funzione libera

```
template<typename T> // Definizione di un template per supportare diversi
tipi numerici (int, long, ecc.)
rational<T> operator+( // Definizione dell'operatore + che restituisce
un nuovo oggetto rational<T>
rational<T> const& a, // Primo operando (sinistra), passato per riferimento

costante per efficienza
rational<T> const& b  // Secondo operando (destra), passato per riferimento
costante
) {
return rational<T>{ // Creazione e ritorno di un nuovo oggetto temporaneo
(RVO - Return Value Optimization)
```

```
a.numerator() * b.denominator() + b.numerator() * a.denominator(), //



Calcolo del nuovo numeratore
(ad + bc)
a.denominator() * b.denominator() //

Calcolo del nuovo denominatore comune (bd)
};
}
```

Esempio di Utilizzo

```
rational<int> r1{1, 2}; // Rappresenta 1/2
rational<int> r2{1, 3}; // Rappresenta 1/3
rational<int> sum = r1 + r2; // Il compilatore chiama operator+(r1, r2)
// Risultato atteso: 5/6
```

3.2.1 Spiegazione Passo-Passo del Codice

1. **La parola chiave** `operator`: Indica al compilatore che stiamo definendo il comportamento di un simbolo specifico (+). Senza questa parola chiave, il compilatore tratterebbe il simbolo come un errore di sintassi.

2. **Parametri** `const&`: L'uso del riferimento (`&`) evita la copia dell'intero oggetto, risparmiando memoria. Il qualificatore `const` garantisce che la funzione non modifichi gli addendi originali, rispettando la semantica matematica della somma (che produce un nuovo valore senza alterare i precedenti).

3. **Accesso tramite Metodi**: Si noti l'uso di `a.numerator()` e `a.denominator()`. Questo suggerisce che la funzione è definita all'esterno della classe (funzione libera) e accede ai dati tramite l'interfaccia pubblica della classe, favorendo l'incapsulamento.

4. **Logica Matematica**:

   - Il nuovo numeratore viene calcolato moltiplicando il numeratore di uno per il denominatore dell'altro e sommandoli.
   - Il nuovo denominatore è il prodotto dei due denominatori originali.

5. **Ritorno per Valore**: La funzione restituisce un oggetto `rational<T>` per valore. Questo è corretto poiché il risultato della somma è un valore nuovo che non esisteva prima della chiamata.

3.3 Osservazione sulla Risoluzione delle Chiamate

Quando il compilatore incontra l'espressione `r1 + r2`, esegue una ricerca (denominata *Argument Dependent Lookup* o ADL) per trovare una funzione chiamata `operator+` che accetti due argomenti di tipo `rational`. Una volta trovata, sostituisce l'espressione con la chiamata:

```
operator+(r1, r2);
```

4 Perché Sovraccaricare gli Operatori?

L'obiettivo primario dell'overloading degli operatori non è la creazione di una sintassi "sofisticata" o puramente estetica, ma il raggiungimento della massima chiarezza espressiva. In un contesto di sviluppo software professionale, il codice viene letto molto più spesso di quanto venga scritto; pertanto, l'uso di una notazione naturale riduce drasticamente lo sforzo cognitivo necessario per comprendere la logica di un algoritmo.

4.1 Confronto: Funzioni vs Operatori

Senza l'overloading degli operatori, per manipolare oggetti definiti dall'utente saremmo costretti a utilizzare chiamate a funzioni esplicite. Questo approccio, sebbene funzionale, degrada rapidamente la leggibilità quando le operazioni diventano nidificate.

4.1.1 Approccio Standard (Senza Overloading)

Si consideri la necessità di eseguire un'operazione matematica tra due oggetti a e b. Dovremmo scrivere:

```
add(a, b)
```

Se l'operazione diventasse più complessa, ad esempio (a+b)× c, la sintassi diventerebbe:

```
multiply(add(a, b), c)
```

In questo scenario, la struttura logica della formula matematica viene oscurata dalla gerarchia delle chiamate di funzione.

4.1.2 Approccio con Overloading

Grazie all'overloading, la stessa operazione viene scritta come:

```
(a + b) * c
```

Questa notazione permette ai tipi definiti dall'utente di comportarsi esattamente come oggetti matematici. Il programmatore può concentrarsi sulla formula stessa anziché sulla meccanica delle chiamate.

4.2 Obiettivi Didattici e Professionali

L'adozione di questa tecnica mira a tre traguardi fondamentali:

1. **Chiarezza**: Il codice riflette direttamente le specifiche del dominio (se sto scrivendo un software di fisica, voglio vedere equazioni, non catene di metodi).
2. **Notazione Naturale**: Gli sviluppatori hanno familiarità con gli operatori aritmetici sin dai primi anni di istruzione. Sfruttare questa conoscenza pregressa rende il codice più intuitivo.
3. **Astrazione**: Permette di nascondere i dettagli implementativi (come la gestione dei puntatori o dei denominatori comuni) dietro un simbolo semplice e universale.

Osservazione: L'overloading deve essere usato solo quando il significato dell'operatore è ovvio. Sovraccaricare l'operatore + per eseguire un'operazione che non sia semanticamente riconducibile a una somma è considerata una pessima pratica di programmazione.

4.3 Esempio di Codice Comparativo

Analizziamo come la stessa operazione cambi radicalmente forma tra una gestione basata su funzioni e una basata su operatori.

Listing 4: Confronto tra chiamate a funzione e operatori

```
// Definizione di una classe semplificata per numeri Complessi
struct Complesso {
double reale, immag; // Parte reale e immaginaria

// Metodo esplicito per la somma
static Complesso somma(const Complesso& a, const Complesso& b) {
// Funzione statica
    return {a.reale + b.reale, a.immag + b.immag}; // Ritorna un nuovo ogge
}

// Overloading dell'operatore +
Complesso operator+(const Complesso& b) const { // Funzione operatore
    return {reale + b.reale, immag + b.immag}; // Ritorna la somma
}
};
```

```
int main() {
Complesso c1{1.0, 2.0}, c2{2.0, 3.0}, c3{0.5, 0.5}; // Tre numeri
complessi

// Utilizzo SENZA overloading: sintassi pesante e poco intuitiva
Complesso ris1 = Complesso::somma(Complesso::somma(c1, c2), c3);
// Somma nidificata

// Utilizzo CON overloading: sintassi naturale e leggibile
Complesso ris2 = c1 + c2 + c3; // Somma lineare immediata

return 0; // Fine del programma
}
```

Analisi dell'Esecuzione

1. **Calcolo di `ris1`**: L'occhio umano deve "decodificare" l'ordine delle chiamate partendo dall'interno. È facile commettere errori di parentesi.
2. **Calcolo di `ris2`**: La leggibilità è istantanea. Grazie alla proprietà associativa dell'operatore `+`, il compilatore valuta prima `c1 + c2` e poi somma `c3` al risultato temporaneo.
3. **Efficienza**: Entrambi i metodi producono codice macchina quasi identico; il vantaggio dell'overloading è puramente umano e architettonico.

5 Operatori Membri vs Operatori Non-Membri

In C++, la progettazione di un operatore sovraccaricato offre due strade implementative principali: definire l'operatore come una funzione membro (all'interno della classe) o come una funzione non-membro (funzione libera o free function). La scelta tra queste due opzioni non è arbitraria, ma incide sulla simmetria delle operazioni e sull'incapsulamento.

5.1 Operatori come Funzioni Membro

Quando un operatore è definito come membro di una classe, l'oggetto alla sinistra dell'operatore è colui che "invoca" la funzione.

- L'operando di sinistra è implicitamente l'oggetto corrente (`*this`).
- L'operando di destra viene passato come argomento della funzione.
- Esempio sintattico: `a + b` viene interpretato dal compilatore come `a.operator+(b)`.

5.2 Operatori come Funzioni Non-Membri (Libere)

Gli operatori definiti come funzioni libere non appartengono a nessuna classe. Entrambi gli operandi vengono passati esplicitamente come argomenti.

- Vengono spesso utilizzati per gli operatori aritmetici binari (come `+`, `-`, `*`, `/`).
- Consentono la simmetria: permettono conversioni implicite su entrambi i lati dell'operatore (ad esempio, sommare un intero a un oggetto razionale sia come `int + rational` che come `rational + int`).
- Esempio sintattico: `a + b` viene interpretato come `operator+(a, b)`.

5.3 Operatori di Confronto

Gli operatori di confronto (`==`, `!=`, `<`, `>`, `<=`, `>=`) seguono spesso la stessa logica degli operatori aritmetici e vengono frequentemente implementati come funzioni non-membri per garantire che il confronto tra due oggetti (o tra un oggetto e un tipo compatibile) sia coerente indipendentemente dall'ordine.

- **Eguaglianza (`==`)**: Verifica se due stati logici sono identici.
- **Relazione d'ordine (`<`)**: Necessaria per algoritmi di ordinamento.

Regola d'Oro: Ogni operatore sovraccaricato deve rispettare il significato logico dell'operazione originale. Sovraccaricare `<` per eseguire un'azione che non sia un confronto d'ordine violerebbe il "Principio della Minima Sorpresa", rendendo il codice prono a bug e difficile da manutenere.

5.4 Implementazione Tecnica e Confronto

Il seguente codice mostra la differenza sintattica e logica tra le due implementazioni applicate a una classe `Numero`.

Listing 5: Confronto tra Operatori Membri e Non-Membri

```
#include <iostream>

class Numero {
private:
int valore; // Dato membro privato

public:
// Costruttore esplicito
explicit Numero(int v) : valore(v) {} // Inizializza il membro 'valore'

// Getter per accedere al valore privato (usato dalla funzione libera)
int getValore() const { return valore; } // Ritorna il valore interno

// 1. Esempio di Operatore Membro (Unario: Cambio di segno)
Numero operator-() const {
    return Numero(-valore); // Ritorna un nuovo oggetto col segno invertito
}
};

// 2. Esempio di Operatore NON-Membro (Binario: Somma)
// Definito fuori dalla classe per garantire simmetria
Numero operator+(const Numero& a, const Numero& b) {
// Accede ai dati tramite l'interfaccia pubblica (getValore)
return Numero(a.getValore() + b.getValore()); // Somma i valori interni
}

// 3. Operatore di confronto NON-Membro
bool operator==(const Numero& a, const Numero& b) {
// Ritorna vero se i valori contenuti sono identici
return a.getValore() == b.getValore();
}

int main() {
Numero n1{10}; // Istanza con valore 10
```

```
Numero n2{20}; // Istanza con valore 20

// Chiamata operatore binario non-membro
Numero n3 = n1 + n2; // Equivale a operator+(n1, n2)

// Chiamata operatore unario membro
Numero n4 = -n1;      // Equivale a n1.operator-()

// Confronto
if (n1 == n2) {       // Equivale a operator==(n1, n2)
    std::cout << "Uguali" << std::endl; // Riga eseguita se uguali
} else {
    std::cout << "Diversi" << std::endl; // Riga eseguita se diversi
}

return 0; // Termine esecuzione
}
```

Output Atteso

```
Diversi
```

5.4.1 Analisi Passo-Passo

1. **Operatore Unario** (`operator-`): È implementato come membro perché agisce su un solo oggetto. La sintassi `-n1` è naturale e non richiede simmetria con altri tipi.

2. **Operatore Binario** (`operator+`): Implementato come non-membro. Se avessimo una conversione implicita da `int` a `Numero`, la funzione libera permetterebbe di scrivere sia `n1 + 5` che `5 + n1`. Se fosse stato un membro, `5 + n1` sarebbe fallito perché l'intero `5` non possiede un metodo `.operator+()`.

3. **Incapsulamento**: La funzione libera `operator+` utilizza `getValore()`. Se la funzione avesse bisogno di accedere direttamente ai membri privati senza usare i getter, dovrebbe essere dichiarata come `friend` all'interno della classe.

4. **Logica di Confronto**: L'operatore `==` restituisce un `bool`, rispettando l'aspettativa che un confronto produca un risultato booleano e non un nuovo oggetto `Numero`.

6 Buone Pratiche nell'Overloading degli Operatori

L'overloading degli operatori è uno strumento estremamente potente che, se usato impropriamente, può rendere il codice criptico e difficile da manutenere. Per questo motivo, la progettazione degli operatori deve seguire principi rigorosi di ingegneria del software, garantendo che l'astrazione creata sia coerente con le aspettative dell'utente e con le leggi matematiche del dominio trattato.

6.1 Principi Guida per una Progettazione Corretta

Un overloading ben progettato deve rispettare i seguenti quattro pilastri fondamentali:

- **Preservare il significato intuitivo**: L'operatore deve eseguire l'azione che il suo simbolo suggerisce universalmente. Ad esempio, l'operatore `+` dovrebbe sempre rappresentare una forma di addizione o concatenazione, mai una sottrazione o una cancellazione di dati.

- **Non modificare la precedenza degli operatori**: In C++, la precedenza e l'associatività degli operatori (ad esempio, il fatto che la moltiplicazione `*` venga eseguita prima dell'addizione `+`) sono cablate nel linguaggio e non possono essere modificate. È essenziale che la logica dell'utente non cerchi di "forzare" comportamenti che violino queste gerarchie sintattiche.

- **Evitare comportamenti sorprendenti**: L'utente si aspetta che gli operatori si comportino come i loro equivalenti per i tipi *built-in*. Se `a + b` modifica il valore di `a`, il programmatore rimarrà sorpreso, poiché l'addizione standard non è distruttiva. Comportamenti inattesi sono la causa principale di bug logici difficili da individuare.

- **Mantenere gli invarianti di classe**: Ogni operazione deve garantire che l'oggetto risultante sia in uno stato valido. Ad esempio, in una classe `rational`, l'operatore di divisione deve gestire o prevenire la creazione di una frazione con denominatore zero.

6.2 L'Esempio del Tipo Razionale

Se implementiamo una classe `rational<T>` per rappresentare i numeri razionali (Q), l'overloading dell'operatore `+` non è solo una scelta sintattica, ma un atto di modellazione matematica.

L'espressione `a + b` per due oggetti `rational` deve produrre un risultato che rispetti le proprietà dell'addizione nel campo dei numeri razionali:

1. **Chiusura**: Il risultato di `rational + rational` deve essere un `rational`.
2. **Commutatività**: `a + b` deve restituire lo stesso valore di `b + a`.
3. **Elemento Neutro**: Sommare una frazione che rappresenta lo zero non deve alterare l'oggetto originale.

6.3 Esempio di Codice: Implementazione "Safe" e Coerente

Il seguente esempio mostra come implementare l'operatore di addizione e l'operatore di assegnamento con addizione (`+=`) mantenendo la coerenza logica e la sicurezza dei dati.

Listing 6: Esempio di buone pratiche: operatori += e +

```
#include <iostream>
#include <numeric> // Per std::gcd (massimo comun divisore)

template <typename T>
class rational {
private:
T num, den; // Invariante: den deve essere sempre != 0

// Funzione di utilita' per mantenere l'invariante (semplificazione)
void simplify() {
    if (den == 0) throw std::runtime_error("Denominatore zero!");
    // Controllo sicurezza
    T common = std::gcd(num, den); // Calcola il MCD
    num /= common; // Divide il numeratore per il MCD
    den /= common; // Divide il denominatore per il MCD
}
public:
rational(T n, T d) : num(n), den(d) { simplify(); } // Costruttore
che semplifica
```

```
// Buona pratica: implementare prima l'operatore composto +=
rational& operator+=(const rational& other) {
    // Calcolo: n1/d1 + n2/d2 = (n1*d2 + n2*d1) / (d1*d2)
    num = num * other.den + other.num * den; // Aggiorna il
    numeratore corrente
    den = den * other.den; // Aggiorna il denominatore corrente
    simplify(); // Ripristina l'invariante (semplificazione)
    return *this; // Ritorna l'oggetto modificato per riferimento
}

// Getter per la stampa
void print() const { std::cout << num << "/" << den; }
};

// Buona pratica: implementare l'operatore binario + usando +=
template <typename T>
rational<T> operator+(rational<T> lhs, const rational<T>& rhs) {
lhs += rhs; // Riutilizza la logica di += sull'oggetto lhs (passato per
copia)
return lhs; // Ritorna il nuovo oggetto risultante
}

int main() {
try {
rational<int> a{1, 2}; // Crea 1/2
rational<int> b{1, 3}; // Crea 1/3

    rational<int> c = a + b; // Utilizza l'operatore +
    (non modifica a o b)

    std::cout << "Risultato di "; a.print();
    std::cout << " + "; b.print();
    std::cout << " = "; c.print();
    std::cout << std::endl;
} catch (const std::exception& e) {
    std::cerr << "Errore: " << e.what() << std::endl; // Gestisce eccezioni
}
return 0;
}
```

Analisi dell'Esecuzione e delle Buone Pratiche

1. **Invarianti**: La funzione `simplify()` viene chiamata dopo ogni operazione. Questo assicura che la frazione sia sempre ai minimi termini e che il denominatore non sia mai zero, mantenendo la validità logica dell'oggetto.

2. **Riutilizzo del Codice**: L'operatore `+` è implementato invocando l'operatore `+=`. Questa è una tecnica standard: si definisce la logica di modifica (`+=`) come membro, e si definisce l'operatore binario (`+`) come funzione libera che lavora su una copia. Ciò garantisce che `+` e `+=` siano sempre coerenti tra loro.

3. **Passaggio per Valore Strategico**: Nell'operatore `+`, l'argomento `lhs` (left-hand side) è passato per valore. Questo crea automaticamente la copia necessaria su cui applicare `+=`, rendendo il codice elegante ed efficiente (sfruttando la *copy elision* del compilatore).

4. **Coerenza Matematica**: L'operazione non produce effetti collaterali inattesi sugli operandi originali, modellando correttamente l'astrazione dei numeri reali.

7 Overloading dell'Operatore di Output <<

L'operatore di inserimento nel flusso (*stream insertion operator*), rappresentato dal simbolo «, è uno degli operatori più frequentemente sovraccaricati in C++. Il suo scopo primario è definire come un oggetto di una classe definita dall'utente debba essere rappresentato testualmente quando viene inviato a un flusso di output, come `std::cout` o un file.

7.1 Perché l'Operatore di Output deve essere una Funzione Libera?

A differenza di altri operatori, l'operatore « non può essere un metodo membro della classe che stiamo creando (ad esempio `rational`). Il motivo risiede nella sintassi dell'operazione binaria:

- In un'espressione come `std::cout « r`, l'operando di sinistra è di tipo `std::ostream` (il flusso), mentre quello di destra è il nostro oggetto `rational`.
- Se fosse un metodo membro, dovrebbe appartenere alla classe `std::ostream`. Poiché non possiamo modificare la libreria standard di C++, dobbiamo implementarlo come una funzione libera (non-membro).

7.2 Analisi della Firma della Funzione

La firma standard per l'overloading di questo operatore è la seguente:
std::ostream& operator«(std::ostream& os, const rational& r)

- `std::ostream&` **(Tipo di ritorno)**: La funzione restituisce un riferimento allo stream di output. Questo è fondamentale per permettere il cosiddetto *chaining* (concatenazione), ovvero scrivere istruzioni come `std::cout « r1 « " e " « r2;`.
- `std::ostream& os` **(Primo parametro)**: Rappresenta il flusso di destinazione. Viene passato per riferimento perché lo stream viene modificato durante l'operazione di scrittura.
- `const rational& r` **(Secondo parametro)**: È l'oggetto che vogliamo stampare. Viene passato per riferimento costante per evitare una copia inutile e per garantire che la stampa non modifichi l'oggetto originale.

7.3 Implementazione Dettagliata

Vediamo come implementare correttamente l'operatore per la classe `rational`, assicurandoci di seguire la formattazione richiesta.

Listing 7: Overloading dell'operatore « per la classe rational

```
#include <iostream>

class rational {
private:
int num, den; // Membri privati per numeratore e denominatore

public:
rational(int n, int d) : num(n), den(d) {} // Costruttore semplice

// Metodi "getter" per accedere ai membri privati
int numerator() const { return num; }       // Ritorna il numeratore
int denominator() const { return den; }     // Ritorna il denominatore
};

// Implementazione dell'operatore come funzione libera
std::ostream& operator<<(std::ostream& os, const rational& r) {
os << r.numerator()    // Inserisce il numeratore nel flusso
<< '/'               // Inserisce il carattere di separazione per la frazione
<< r.denominator(); // Inserisce il denominatore nel flusso

return os;             // Ritorna lo stream per permettere la
concatenazione
}

int main() {
rational r{3, 4};      // Creazione di un oggetto rational (3/4)

// Utilizzo dell'operatore sovraccaricato
std::cout << "La frazione e': " << r << std::endl;

return 0;              // Fine del programma
}
```

Output Atteso

```
La frazione e': 3/4
```

7.3.1 Spiegazione Passo-Passo dell'Esecuzione

1. **Chiamata all'Operatore**: Quando il compilatore incontra `std::cout « r`, cerca una funzione `operator«` che accetti un `std::ostream` a sinistra e un `rational` a destra.

2. **Flusso di Dati**: All'interno della funzione, i dati dell'oggetto `r` (ottenuti tramite i metodi `numerator()` e `denominator()`) vengono inviati all'oggetto `os` (che in questo caso è un riferimento a `std::cout`).

3. **Il Ruolo del Carattere `'/'`**: Il carattere viene inserito letteralmente tra i due numeri per creare la rappresentazione visiva standard di una frazione.

4. **L'Importanza del `return os`**: Restituendo il riferimento allo stream, l'espressione `(std::cout « r)` "diventa" nuovamente `std::cout`. Questo permette di applicare immediatamente l'operatore successivo, come ad esempio `« std::endl`.

Nota di Progettazione: Se la classe `rational` non fornisse metodi pubblici come `numerator()`, l'operatore dovrebbe essere dichiarato come `friend` all'interno della classe per poter accedere direttamente ai membri privati `num` e `den`. Tuttavia, l'uso dei getter è generalmente preferito per mantenere un incapsulamento più pulito.

8 Anatomia della Firma dell'Operatore di Output

Per comprendere appieno come il C++ gestisca l'interazione tra gli oggetti e i flussi di dati (stream), è necessario analizzare nel dettaglio ogni componente della firma della funzione di overloading dell'operatore «. La firma standard, come abbiamo visto, è:

```
std::ostream& operator«(std::ostream& os, rational const& r)
```

Questa struttura non è casuale, ma risponde a precise regole del linguaggio e necessità di efficienza.

8.1 Analisi dei Componenti della Firma

Ogni elemento della dichiarazione svolge un ruolo cruciale nel garantire che l'operatore si comporti in modo coerente con il resto della libreria standard.

- **Funzione Non-Membro**: Come specificato in precedenza, l'operatore deve essere una funzione libera. Questo perché l'operando di sinistra non è un oggetto della classe `rational`, ma un oggetto della classe `std::ostream`. Definendola come funzione non-membro, il compilatore può far corrispondere i due tipi di argomenti (`ostream` e `rational`) correttamente.
- **L'Operando di Sinistra (`std::ostream& os`)**: Rappresenta il "canale" di destinazione. Viene passato per **riferimento** (`&`) perché un oggetto di tipo stream non può essere copiato e, durante l'operazione di scrittura, il suo stato interno (es. la posizione del cursore o eventuali flag di errore) viene modificato.
- **L'Operando di Destra (`rational const& r`)**: Rappresenta l'oggetto che deve essere "letto" per produrre l'output.
  - **Uso di `const`**: Garantisce che l'operazione di stampa non alteri in alcun modo l'oggetto originale. La stampa è, per definizione, un'operazione di sola lettura.
  - **Uso del riferimento (`&`)**: Evita di creare una copia locale dell'oggetto `rational` nello stack della funzione, migliorando le prestazioni, specialmente se l'oggetto contiene molti dati.
- **Tipo di Ritorno (`std::ostream&`)**: La funzione restituisce lo stesso stream che ha ricevuto in input. Questo permette il concatenamento delle operazioni (es. `cout « r1 « r2;`). Senza questo ritorno, potremmo stampare un solo oggetto per riga di codice.

8.2 Implementazione e Flusso Logico

Il compito della funzione è quello di "iniettare" una rappresentazione testuale dell'oggetto nel flusso fornito.

Listing 8: Dettaglio implementativo della firma dell'operatore «

```
#include <iostream>

// Supponiamo che rational sia gia definito con i metodi numerator()
e denominator()

std::ostream& operator<<( // Ritorna un riferimento allo stream per
il "chaining"
std::ostream& os,      // Primo parametro: lo stream di
destinazione (es. cout)
const rational& r      // Secondo parametro: l'oggetto da
visualizzare (sola lettura)
) {
// Inseriamo i componenti nel flusso usando gli operatori <<
gia definiti per int e char
os << r.numerator() << '/' << r.denominator();

return os;             // Restituiamo il riferimento allo stream ricevuto
}

int main() {
rational r{5, 9};      // Istanza della classe rational

// Il compilatore trasforma la riga seguente in: operator<<(std::cout, r)
std::cout << r << "␣e'␣una␣frazione." << std::endl;

return 0;
}
```

Analisi Passo-Passo dell'Esecuzione

1. **Risoluzione del Simbolo**: All'incontro di `std::cout « r`, il compilatore identifica che il primo argomento è un `ostream` e il secondo è un `rational`. Cerca quindi la funzione corrispondente.

2. **Esecuzione del Corpo**: La funzione accede alle proprietà di `r`. Invia al parametro `os` (che punta a `cout`) il valore numerico del numeratore, poi il carattere '/', e infine il denominatore.

3. **Restituzione del Controllo**: La funzione termina restituendo `os`. A questo punto, l'istruzione nel `main` diventa virtualmente `std::cout « " e' una frazione."`.

4. **Concatenazione Successiva**: Il processo si ripete per la stringa e per `std::endl`, grazie al fatto che ogni chiamata restituisce lo stream necessario per la successiva.

8.2.1 Osservazione sui Prerequisiti

È importante notare che affinché questa implementazione funzioni, i tipi restituiti da `r.numerator()` e `r.denominator()` (solitamente `int` o `long`) devono a loro volta avere l'operatore « definito. Poiché si tratta di tipi primitivi, il C++ fornisce già queste versioni dell'operatore nella libreria `<iostream>`.

9 Perché restituire `std::ostream&`?

L'ultimo passaggio fondamentale nell'implementazione dell'operatore di output è l'istruzione `return os;`. Questa scelta progettuale non è opzionale, ma è il requisito tecnico che permette al C++ di gestire flussi di dati in modo fluido e naturale. Restituendo lo stesso oggetto stream che è stato ricevuto come argomento, la funzione permette il meccanismo del concatenamento (operator chaining).

9.1 Il Meccanismo del Concatenamento

In C++, è prassi comune scrivere istruzioni di output che coinvolgono più oggetti sulla stessa riga, come ad esempio:

```
std::cout « r1 « " " « r2;
```

Per capire perché il tipo di ritorno sia cruciale, dobbiamo analizzare come il compilatore interpreta questa riga. Gli operatori `«` hanno un'associatività da sinistra a destra. Questo significa che l'espressione viene valutata un pezzo alla volta, partendo dalla sinistra:

1. Viene valutata la prima parte: `operator«(std::cout, r1)`.
2. Questa chiamata esegue la stampa di `r1` e, grazie a `return os;`, restituisce un riferimento a `std::cout`.
3. L'espressione originale ora "diventa" virtualmente: `(std::cout) « " " « r2;`.
4. Viene quindi chiamata la funzione per la stringa: `operator«(std::cout, " ")`.
5. Anche questa restituisce `std::cout`, rendendo possibile l'ultima valutazione: `std::cout « r2;`.

9.2 Conseguenze della Mancata Restituzione

Se la funzione fosse definita con un tipo di ritorno `void` (ovvero non restituisse nulla), il concatenamento fallirebbe drasticamente.

- **Errore di Compilazione**: Dopo la prima operazione `std::cout « r1`, il compilatore non avrebbe più un oggetto a sinistra per l'operatore successivo.
- **Sintassi Limitata**: Il programmatore sarebbe costretto a scrivere ogni stampa su una riga separata, rendendo il codice inutilmente prolisso e meno leggibile.

9.3 Analisi del Codice e Flusso di Ritorno

Vediamo nel dettaglio cosa accade all'interno della memoria durante queste chiamate.

Listing 9: L'importanza del ritorno del riferimento allo stream

```
#include <iostream>

struct rational {
int n, d; // Numeratore e denominatore
};

// Definizione corretta con ritorno di std::ostream&
std::ostream& operator<<(std::ostream& os, const rational& r) {
os << r.n << "/" << r.d; // Esegue l'operazione di inserimento
nello stream
return os;                // Restituisce lo STESSO oggetto ricevuto in
input
}

int main() {
rational r1{1, 2}; // Frazione 1/2
rational r2{3, 4}; // Frazione 3/4

// Esempio di concatenamento (Chaining)
// Passaggio 1: operator<<(std::cout, r1) -> stampa "1/2" e
ritorna std::cout
// Passaggio 2: lo stream ritornato viene usato per << " e "
// Passaggio 3: lo stream ritornato viene usato per << r2
std::cout << r1 << " e " << r2 << std::endl;

return 0; // Termine programma
}
```

Output Atteso

```
1/2 e 3/4
```

9.3.1 Spiegazione Passo-Passo dell'Infrastruttura di Ritorno

1. **Il Riferimento (&)**: Notate che il tipo di ritorno è `std::ostream&` (un riferimento). Non stiamo creando una copia di `std::cout` (operazione che sarebbe impossibile, dato che i flussi non sono copiabili), ma stiamo restituendo un "alias" all'oggetto originale che risiede nella memoria di sistema.

2. **Efficienza del Ritorno**: Restituire un riferimento è un'operazione istantanea e a costo zero per la CPU, poiché non comporta spostamento di dati pesanti.

3. **L'oggetto `os`**: All'interno della funzione, `os` è il nome locale per lo stream passato (es. `std::cout`). Quando scriviamo `return os;`, stiamo dicendo al programma: "Ho finito di usare questo stream per la mia classe, ora ridallo al chiamante così può usarlo per qualcos'altro".

4. **Il Risultato Finale**: Grazie a questa convenzione, le classi definite dall'utente si integrano perfettamente nell'ecosistema della libreria standard, garantendo un'esperienza di programmazione uniforme.